
FiReplace

A wireless replacement control system for a gas fireplace appliance — design & build plan

A from-scratch DIY control system for a gas-powered fireplace appliance, built from three coordinating ESP32 nodes — a mains-adjacent Main Control Block, a battery-powered handheld Remote, and a coin-cell wireless temperature Sensor — linked over ESP-NOW with an optional MQTT bridge for home-server control. Designed as a hobby build first, with hardware and firmware choices kept product-viable throughout.

Design document · synthesized from project planning · guidelines, not a strict requirements list
Copyright 2026 — Noah Robertson

Contents

1. Overview
2. Background & motivation
3. Design decisions
4. System architecture
5. Hardware requirements
6. Fireplace interface circuit
7. Firmware architecture
8. Output precedence logic
9. Configuration model
10. Pairing & sync protocol
11. ESP-IDF project structure
12. PCB design
13. Indicative bill of materials
14. Plan of attack
15. Safety notice
16. Appendix: pin budget & references

1 Overview

FiReplace replaces a broken third-party remote-control system for a gas-powered fireplace appliance with a purpose-built, three-device wireless system: a **Main Control Block** that drives the fireplace and hosts an optional home-server bridge, a handheld **Remote Control**, and a standalone wireless **Sensor**. All three are ESP32-family devices linked by ESP-NOW, chosen specifically because the whole system is expected to eventually pass through Espressif's WiFi/BLE radio regardless of device — see [Section 3](#).

The three devices split cleanly by role. The Main Control Block is USB-C powered and does the real work: it drives the fireplace's on/off control leads, runs the AUTO thermostat logic, and optionally bridges to a home server over WiFi/MQTT. The Remote is a 2×AAA handheld with four buttons and a press-to-view display that sends short commands. The Sensor is a coin-cell device that periodically wakes, takes a reading, and reports it — it does not need to be polled.

This document is written as a set of **guidelines, not a strict requirements list**. Several specifics (exact pulse timing into the fireplace's control interface, WiFi provisioning flow, enclosure material) are flagged explicitly as assumptions to verify on the bench rather than settled requirements.

2 Background & motivation

The project exists to replace a bad/broken existing fireplace remote-control system — hence the name. The fireplace appliance itself is controlled by a simple bipolar low-voltage interface: driving its two control leads to roughly **+5-6 V turns it on**, and reversing to roughly **−5-6 V turns it off**. This is consistent with common aftermarket millivolt-system fireplace receivers, which present a low-voltage command interface to the appliance's own thermopile-powered gas valve electronics — the FiReplace boards never touch mains or the gas valve itself.

Three requirements shape every hardware and firmware decision that follows:

- **Low power on the two field devices.** The Remote and Sensor need to run for months to years on a single set of batteries; the Main Control Block, being USB-powered, is not power-constrained.
- **Zero-configuration pairing.** The Remote and Sensor must connect to the Main Control Block automatically out of the box, with no app or manual configuration step.
- **General-purpose firmware.** The system should be designed so the hardware could later drive something other than a fireplace, without a from-scratch rewrite.

A fourth, unstated requirement runs through the whole design: this is a **DIY build that may become a small product**. Where a DIY-only project might cut a corner, this document tries to flag it rather than silently take the shortcut — see the callout boxes throughout, and [Section 15](#).

3 Design decisions

The following choices were made during the design session; they shape every later section.

Decision	Choice	Rationale
Radio link	ESP-NOW, all-ESP32 devices	Connectionless, no router required, MAC-based addressing supports auto-pairing. Keeping every device on Espressif silicon means one radio stack and one debugging toolchain across all three boards.
Main MCU	ESP32-S3-WROOM-1	Native USB on the same USB-C connector already used for power, doubling as the flash/debug port. Dual-core headroom for scheduler + WiFi/MQTT + sensor fusion.
Remote & Sensor MCU	ESP32-C3-MINI-1 (both)	Mature, well-documented low-power RISC-V part with a large reference-design and troubleshooting base. A same-family alternative (ESP32-C2 / ESP8684) was evaluated and rejected for the Sensor — its thinner ecosystem wasn't worth ~\$1 of BOM savings on a DIY build.
Sensor reporting model	Self-report on a timer (push), not polled	An ESP32 cannot receive ESP-NOW while deep-asleep, so a true on-demand "main pages the sensor" model is expensive on a coin cell. The Sensor instead wakes on its own RTC interval, reads, transmits, and sleeps; the Main Control Block caches the latest value with a staleness timeout.
Temperature sensor IC	Sensirion SHT40, used on both Main and Sensor boards	One part number for BOM and firmware commonality. The Main board carries its own onboard SHT40 dedicated to the overtemp failsafe (see below), independent of the Sensor's reading.
Overtemp failsafe path	Onboard sensor + firmware-only lockout	The lockout logic itself is firmware, not a hardware comparator — but it is fed by a sensor physically on the Main board rather than solely trusting the wireless Sensor, so the safety path doesn't silently vanish if the Sensor's coin cell dies or its link drops. See Section 15 for the residual risk this leaves.
Fireplace drive	Single DPDT relay wired as a polarity reverser	One relay, one coil, driven straight off the 5 V USB rail — no boost converter needed. De-energized rest state is inherently fail-safe. See Section 6 .

WiFi/MQTT switch	Runtime-checked GPIO, ESP-NOW remote path always live	The switch only ever gates the optional WiFi/MQTT bridge. The physical Remote's ESP-NOW control path works identically whether the switch is on or off — flipping it only adds or removes the network channel, live, without a reboot.
Remote display	Press-to-view OLED	A small SSD1306-class I2C OLED lit only for a few seconds after a button press. A driven segment LCD (HT1621-class) was also considered — its datasheet current (~0.4–0.6 mA while active) turns out to be a fixed cost comparable to or worse than an OLED's real-world usage-averaged power for a display that's only glanced at briefly, and it would need either a second 5 V rail (off-the-shelf breakouts) or a bare-chip custom circuit (harder to source/assemble) to run cleanly. OLED wins on ecosystem maturity and simplicity for a DIY build; keep the display off between interactions to bound its contribution to battery life.
Sensor placement	Room-ambient, a few feet from the appliance	Sees normal indoor ambient (~10–40°C), so standard commercial-grade (0–70°C) parts and enclosure materials are appropriate — no elevated-temperature derating needed.
v1 expansion headroom	None — exact-fit hardware, general-purpose firmware	"Could drive something else later" is treated as a firmware/abstraction goal for v1, not a hardware commitment (no stubbed spare relay channel or header). A future non-fireplace variant is a deliberate respin.

4 System architecture

Three ESP32 nodes form a star around the Main Control Block. The Remote and Sensor each hold only the Main's MAC address and talk to nothing else. The Main additionally, and optionally, bridges to a home server over WiFi/MQTT — that link is gated by the physical WiFi switch and is never required for local control.

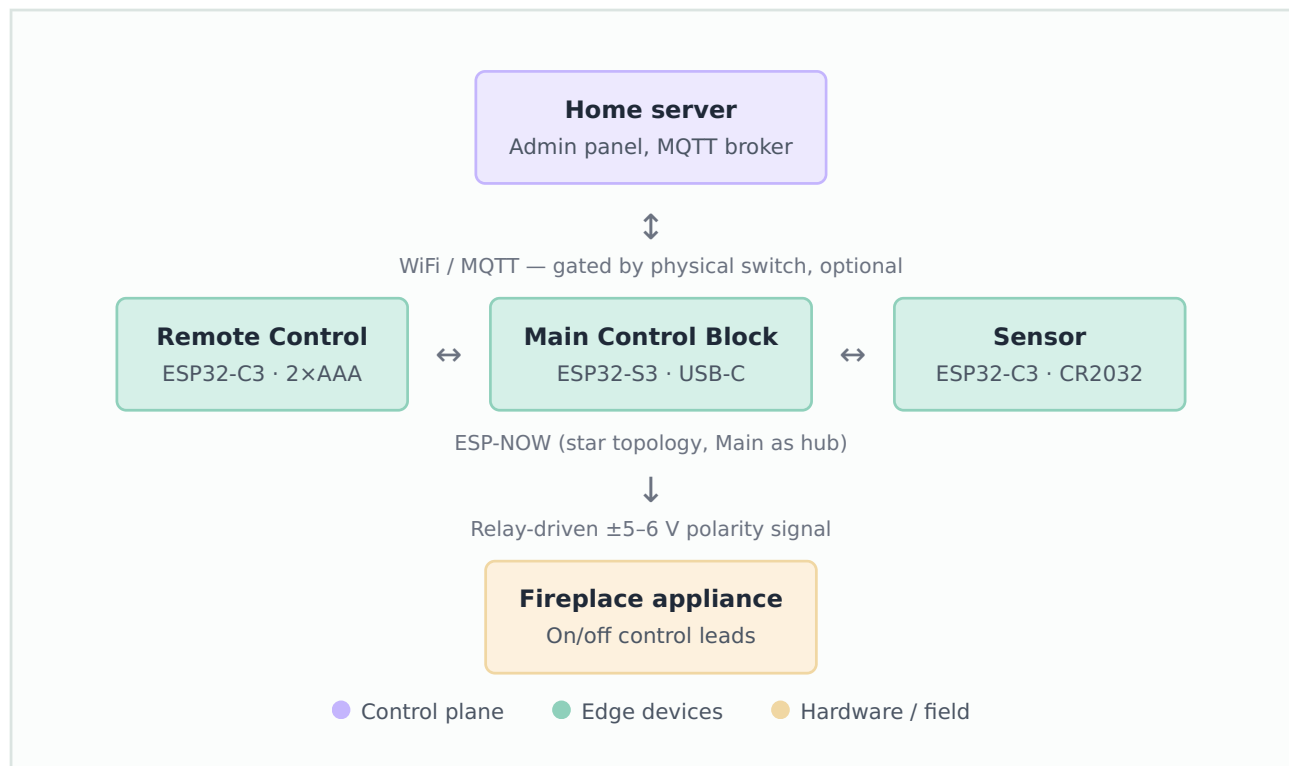


Figure 1 — Three-node star topology with an optional control-plane bridge.

This keeps the safety-relevant path short: a button press on the Remote reaches the Main Control Block over a single ESP-NOW hop, with no dependency on WiFi, a broker, or a home server being up. The MQTT bridge exists purely for convenience (scheduling from a browser, remote status) and is architecturally a peer of the Remote, not a replacement for it — see the precedence rules in [Section 8](#).

5 Hardware requirements

Main Control Block

Signal	Direction	Notes
Onboard temperature (SHT40, I2C)	Input	Dedicated to the overtemp failsafe path; independent of the Sensor's reading.
WiFi-mode switch	Input	Panel toggle, read continuously at runtime (not just at boot). Gates WiFi/MQTT only.
Relay drive (NFET gate)	Output	3.3 V logic into the low-side driver for the DPDT relay coil.
Status LED	Output	Optional; system/WiFi status indicator.
USB-C (native)	Power + debug	5 V power in; native USB for flashing/JTAG/console on the S3.

Remote Control

Signal	Direction	Notes
On/Off button	Input	Toggles the fireplace between AUTO (enabled) and OFF (disabled). Wake source from deep sleep.
Temp + / Temp – buttons	Input	Adjusts the AUTO setpoint; also wakes the display.
°F/°C button	Input	Local display-format toggle only — never transmitted as a device command.
OLED display (I2C)	Output	Press-to-view; powered on for a few seconds after any button press, then off.
Battery voltage (ADC)	Input	Optional low-battery indication on the display.

Sensor

Signal	Direction	Notes
Temperature (SHT40, I2C)	Input	Read once per wake cycle, then the bus is powered down.

The Sensor's entire GPIO budget is essentially one I2C bus — everything else about it is the RTC wake timer and the ESP-NOW radio, both internal to the module.

Compliance note — transmit only on user action or scheduled wake

The doc calls out that the Remote must "send signals only on button press." The Sensor's periodic self-report is a scheduled transmission rather than a user action, but it is still a short, infrequent, low-duty-cycle burst (a fraction of a second every 30–60 s) consistent with typical unlicensed 2.4 GHz duty-cycle practice — confirm against the regulations applicable in your region if this ships as a product.

6 Fireplace interface circuit

The fireplace expects roughly +5–6 V across its two control leads to turn on, and the reverse polarity to turn off. A single DPDT relay, wired so its two poles route the same 5 V USB rail to the two leads in opposite senses, produces exactly this bipolar signal from one unipolar supply — no boost converter or second rail is needed anywhere on the board.

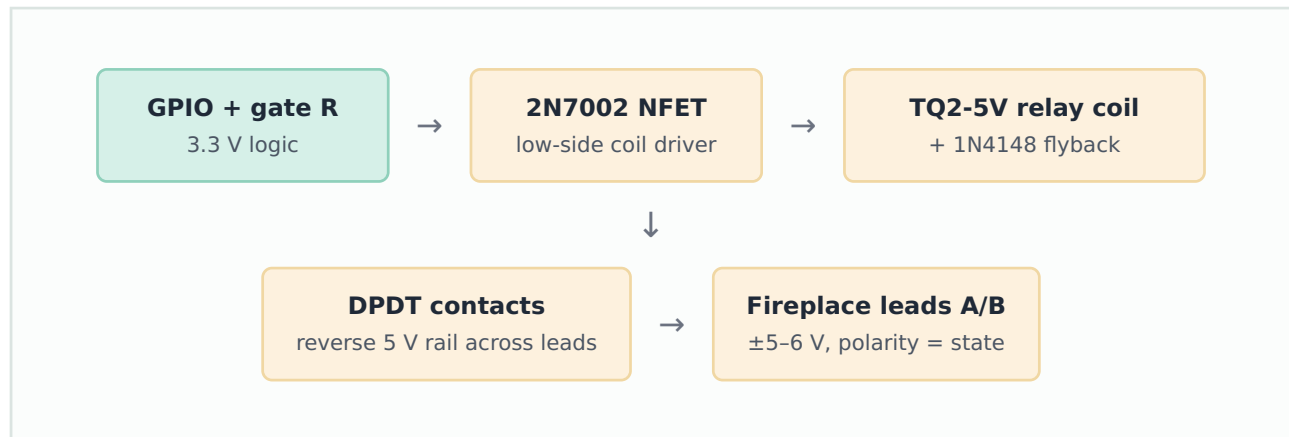


Figure 2 — Relay-based polarity reverser: one coil, one GPIO, no second supply rail.

De-energized (GPIO low, FET off) is defined as the fireplace **off** polarity. This is a meaningful safety property for free: on firmware crash, brownout, or USB power loss, the relay's spring returns it to the de-energized position and the fireplace sees the off-polarity signal with no code running at all.

Verify pulse behavior against your specific appliance

This design assumes the appliance's receiver tolerates a continuously-held reverse-polarity idle voltage between transitions, consistent with common aftermarket fireplace receiver interfaces. If bench testing shows your unit instead expects a momentary pulse followed by an open/floating idle line, add a second relay (SPST) in series to fully disconnect the leads except during a brief (e.g. 500 ms–1 s, tune on the bench) transition pulse. Confirm the exact interface behavior for your appliance before wiring it to a live unit.

7 Firmware architecture

Each device runs its own small set of FreeRTOS tasks, sized to its role.

Main Control Block

Task	Responsibility
Link (ESP-NOW)	Handles pairing handshakes, receives Remote commands and Sensor reports, tracks per-peer last-seen time for staleness detection.
Local sense	Polls the onboard SHT40 on a short interval; feeds the overtemp failsafe path only.
Output manager	Resolves every state change through the precedence rules in Section 8 and drives the relay GPIO.
Net (WiFi/MQTT)	Started/stopped live as the WiFi switch GPIO changes; owns the MQTT client and status/command topics when active.
Config store	Persists setpoint, units, mode, and paired-peer MACs to NVS.

Remote Control

Task	Responsibility
Input	Debounces the four buttons; each press is a wake source from deep sleep.
Display	Drives the OLED for a few seconds after any input, then powers it off.
Link (ESP-NOW)	Transmits each command once, waits briefly for an ESP-NOW delivery ack, retries a bounded number of times, then returns to deep sleep regardless of outcome.

Sensor

Task	Responsibility
Wake / sense	RTC timer wakes the device on a configurable interval (default ~60 s), powers the SHT40, takes one reading.
Link (ESP-NOW)	Transmits the reading (best-effort, no retry — the next wake cycle supersedes it), then the device returns to deep sleep.

8 Output precedence logic

The Main Control Block's output manager is where the overtemp failsafe, manual commands, and the AUTO thermostat all collide. As with any system that has more than one source of truth for an output, the priority order is the single most important correctness decision in the firmware. For the relay state, resolve from highest priority to lowest:

1. **Overtemp lockout active → force OFF.** Latched by the onboard SHT40 reading; clears only on an explicit reset (long-press on the Remote, an MQTT command, or a power cycle).
2. **Sensor reading stale → force OFF.** If no self-report has been heard from the wireless Sensor within a configurable timeout (default 5 minutes), AUTO mode has no data to act on — fail to off rather than run on a last-known-good value indefinitely.
3. **Explicit OFF command → force OFF.** From the Remote's On/Off button or an MQTT command.
4. **Explicit FORCE ON command → force ON.** An advanced override (MQTT only in v1, not exposed on the simple Remote), still subject to rules 1 and 2 above — the failsafe and staleness checks are never bypassed by FORCE ON.
5. **AUTO (default) → thermostat logic.** Hysteresis around the configured setpoint, driven by the most recent (non-stale) Sensor reading.

Reported state always reflects resolved state

The Remote's display and any MQTT status topic should reflect what rule actually won, not the raw command — if a user presses "on" but the overtemp lockout is latched, the display should show locked-out/off, not on. This mirrors the same honesty principle used in the espond32 project's switch-precedence design.

9 Configuration model

The Main Control Block owns one small JSON configuration blob in NVS, parsed with ESP-IDF's bundled cJSON. The Remote and Sensor carry almost no configuration of their own beyond the Main's paired MAC address.

```
{
  "version": 3,
  "mode": "auto",           // "auto" | "off" | "force_on"
  "setpoint_c": 21.5,
  "hysteresis_c": 0.5,
  "units": "F",             // display preference only
  "overtemp_limit_c": 45.0,
  "sensor_stale_timeout_s": 300,
  "wifi_enabled": true,
  "peers": {
    "remote_mac": "AA:BB:CC:11:22:33",
    "sensor_mac": "AA:BB:CC:44:55:66"
  }
}
```

The Remote persists only its own paired Main MAC and the last mode/setpoint it displayed (so the screen has something sensible to show immediately after a button press, before a fresh status echo arrives). The Sensor persists only the paired Main MAC — it has no other state to keep.

10 Pairing & sync protocol

ESP-NOW pairing (out-of-the-box auto-connect)

ESP-NOW requires each side to register its peer's MAC address before it can send — there is no built-in discovery. To meet the "connects automatically out of the box" requirement without an app or manual MAC entry, the Main Control Block runs a pairing listener:

1. A factory-fresh Remote or Sensor, finding no paired MAC in NVS, broadcasts an unencrypted `hello {device_type, random_id}` to the ESP-NOW broadcast address on a short retry loop.
2. The Main Control Block always listens for `hello` broadcasts. On receiving one, it registers the sender as an ESP-NOW peer and replies directly with an `ack {main_mac}`.
3. The peripheral receives the ack, stores the Main's MAC in NVS, and never broadcasts `hello` again — from then on it unicasts directly to the paired Main.

Optional hardening for a product build

v1 pairing is intentionally unauthenticated — anyone's freshly-flashed device could theoretically pair to a nearby Main within range during that device's brief unpaired window. For a DIY build this is a non-issue. If this ships as a product, consider gating the pairing listener behind a physical action (e.g. only listen for `hello` for 30 s after a button/reset combo) and/or adding an ESP-NOW LMK pre-shared at manufacture time.

Optional MQTT bridge

Topic	Publisher	Notes
<code>firereplace/status</code>	Main	Retained; resolved mode, setpoint, latest temperature, sensor freshness, lockout state.
<code>firereplace/cmd</code>	Server	Mode/setpoint changes and the FORCE ON override — same command surface the Remote uses.
<code>firereplace/availability</code>	Main	MQTT Last-Will: <code>online / offline</code> .

Because the local ESP-NOW control path never depends on this bridge, the home server (and the WiFi switch itself) can be offline indefinitely without affecting basic fireplace control from the physical Remote.

11 ESP-IDF project structure

Three independent ESP-IDF projects, one per device, each with one component per concern.

```
firereplace-main/
├─ main/          app_main: nvs + i2c + relay gpio + task launch
├─ components/
│  ├─ link/       ESP-NOW pairing, rx, peer staleness tracking
│  ├─ local_sense/ onboard SHT40, overtemp failsafe input
│  ├─ output_manager/ precedence resolver, drives relay gpio
│  ├─ config/     NVS read/write, cJSON schema
│  └─ net/        wifi switch handling + esp-mqtt bridge
└─ CMakeLists.txt
```

```
firereplace-remote/
├─ main/          app_main: nvs + deep-sleep wake handling
├─ components/
│  ├─ input/      4-button debounce + wake source config
│  ├─ display/    OLED driver, press-to-view timeout
│  └─ link/       ESP-NOW pairing + command tx + ack/retry
└─ CMakeLists.txt
```

```
firereplace-sensor/
├─ main/          app_main: nvs + rtc wake timer + deep sleep
├─ components/
│  ├─ sense/      SHT40 driver, single-shot read
│  └─ link/       ESP-NOW pairing + best-effort report tx
└─ CMakeLists.txt
```

Bring-up order follows the plan in [Section 14](#): the Sensor and Remote's `link` components are deliberately identical in protocol (though separately implemented per device role), so getting ESP-NOW pairing working once on the bench de-risks it for both.

12 PCB design

Three small single-domain boards. Unlike a mains-switching design, none of these boards carry line voltage — the Main Control Block only ever sees USB-C 5 V in and the fireplace's own ≤ 6 V control leads out, so there is no isolation barrier, creepage/clearance analysis, or earth-bonding requirement anywhere in this design. This is a meaningfully simpler safety posture than a mains-switching board.

Main Control Block

- **Power:** USB-C 5 V in (populate the 5.1 k Ω CC1/CC2 pulldowns — without them many USB-C source ports won't present 5 V at all). A small buck converter steps 5 V down to 3.3 V for the logic rail; the relay coil and its 5 V-rated flyback path run directly off USB VBUS, unregulated.
- **Relay placement:** keep the coil flyback diode and NFET close to the relay footprint; route the fireplace-lead terminal block away from the ESP32-S3 module's antenna keep-out at the board edge.
- **Onboard SHT40:** place away from the buck converter and relay (self-heating sources) so the overtemp failsafe reading isn't biased by the board's own components.

Remote Control

- **Power:** 2×AAA holder → boost converter → 3.3 V. Keep the boost converter's inductor and feedback network tight per its datasheet application circuit; this is the noisiest part of an otherwise quiet low-power board.
- **Buttons:** panel-mount or PCB-edge tactile switches positioned for the target enclosure; wire each as a dedicated wake-capable GPIO with an internal pull, no external pull resistors needed.

Sensor

- **Power:** CR2032 holder with a charge-limiting resistor (≈ 10 – 100 Ω) into a bulk buffer capacitor at the ESP32-C3's power pins, sized to source the ESP-NOW TX current pulse without the coin cell's internal resistance dragging the rail down mid-transmission — see the callout in [Section 13](#).
- **Layout:** this is the smallest, simplest board in the system — essentially the ESP32-C3 module, the SHT40, the coin-cell holder, and the buffer capacitor. Minimize trace length on anything that isn't asleep 99% of the time.

PCB process

- **Tooling:** KiCad, using Espressif's official KiCad library (github.com/espressif/kicad-libraries) for the WROOM-1 and MINI-1 module footprints.

- **Layers:** 2 layers are sufficient for all three boards — none of them carry the routing density or current levels that would justify 4 layers.
- **Fabrication:** JLCPCB or PCBWay, with optional SMT assembly for the smaller passives (0402/0603) on the Remote and Sensor boards.

13 Indicative bill of materials

Candidate parts researched against current distributor listings and manufacturer datasheets. Treat exact values and part numbers as a starting point to confirm at bring-up, per the doc's guideline framing.

Main Control Block

Ref	Part	Notes
U1	Espressif ESP32-S3-WROOM-1-N8	No-PSRAM variant is sufficient; native USB to J1.
J1	GCT USB4105-GF-A (USB-C receptacle)	Populate CC1/CC2 5.1 k Ω pulldowns — required for the port to present as a device and pull 5 V from most sources.
D1	ST USBLC6-2SC6	D+/D− ESD array; pair with a VBUS-only TVS (e.g. SMAJ5.0A) for extra surge margin on the power pin.
U2	Diodes Inc AP63203WU-7 (5V→3.3V buck)	Chosen over an AP2112 LDO — combined relay-coil + WiFi-TX current (~250–300 mA peak) dissipates too much heat in an LDO's small package; the buck adds one inductor for meaningfully better thermal headroom.
U3	Sensirion SHT40-AD1B-R2	Onboard failsafe temperature sensor, I2C.
K1	Panasonic TQ2-5V (DPDT signal relay)	5 V coil runs directly off USB VBUS. Omron G5V-2-DC5 is a documented, slightly larger/pricier alternative if TQ2 sourcing is difficult.
Q1	onsemi 2N7002 (N-FET)	Low-side relay coil driver; switches cleanly off a 3.3 V GPIO with no base-resistor sizing.
D2	1N4148	Flyback diode across the relay coil.
SW1	SPDT panel toggle	WiFi-mode switch, read at runtime.
J2	2-position low-voltage terminal block	Fireplace control leads A/B.

Remote Control

Ref	Part	Notes
U1	Espressif ESP32-C3-MINI-1-N4	In-package flash; ~5 μ A deep-sleep at chip level (budget real firmware current higher).

BT1	2×AAA holder (series)	~3.0 V fresh, sags to ~2.0–2.2 V near end of life.
U2	TI TPS61099 (boost, 2×AAA → 3.3V)	1µA quiescent current — meaningfully extends standby life given the Remote is asleep nearly 100% of the time. WSON/ WCSP package; needs stencil/reflow, not ideal for freehand hand-soldering.
U3	SSD1306-class 128×32 I2C OLED module	Press-to-view only; powered off between interactions.
SW1–SW4	Panasonic EVQ-PU series (6×6mm)	On/Off, Temp+, Temp–, °F/°C.

Sensor

Ref	Part	Notes
U1	Espressif ESP32-C3-MINI-1-N4	Same part as the Remote — one MCU line to stock across both battery devices.
BT1	CR2032 holder (Keystone 3034 or MPD BHX2-2032-SM)	SMD, 20 mm coin-cell footprint.
U2	Sensirion SHT40-AD1B-R2	Same part as the Main's onboard sensor.
R1	10–100 Ω charge-limit resistor	Between the coin cell and the buffer capacitor.
C1	220–470µF bulk cap, or Eaton KR/KW coin-form supercap	See callout below — value is a bring-up tuning parameter.

The Sensor's buffer capacitor value is not a fixed spec

ESP-NOW TX current can peak 80–180 mA in short bursts, which a bare CR2032's internal resistance struggles to source without significant voltage sag. The standard fix — a charge-limiting resistor into a bulk capacitor sitting right at the MCU's power pins, so the cap sources the pulse instead of the coin cell directly — is well-established practice for CR2032-powered WiFi-radio sensor nodes, but the exact capacitance needs to be validated empirically against this firmware's actual ESP-NOW TX pulse width during bring-up ([Section 14](#), Phase 7).

14 Plan of attack

Firmware is validated on dev boards for all three devices before any custom PCB is designed.

Phase	Work
0 — Toolchain	Install ESP-IDF; flash the blink/hello-world example to an S3 dev board, a C3 dev board (×2 role) to confirm toolchain and USB-JTAG on each target.
1 — Bare I/O	Main: read the onboard SHT40, drive the relay GPIO into an LED (not a real fireplace yet). Remote: debounce the 4 buttons, drive the OLED. Sensor: read the SHT40 once, log over serial.
2 — ESP-NOW pairing	Implement the hello/ack pairing handshake (Section 10) between a C3 dev board and the S3 dev board; confirm it survives a factory-reset/re-pair cycle.
3 — Output manager + precedence	Implement the 5-rule precedence resolver (Section 8) fully offline against hardcoded/simulated inputs before wiring in live sensor data.
4 — Sensor deep sleep + battery math	Get the Sensor's wake-read-transmit-sleep cycle working on the RTC timer; measure actual current draw on the bench (not just datasheet numbers) and project real coin-cell life at the chosen wake interval.
5 — Remote deep sleep + battery math	Same measurement exercise for the Remote's button-wake cycle on 2×AAA through the boost converter.
6 — Relay bring-up on the bench	Build the polarity-reverser circuit (Section 6) on protoboard, verify the ±5–6 V output with a multimeter under both states before it ever touches the real appliance.
7 — Buffer-cap tuning	With real ESP-NOW TX timing measured, size and verify the Sensor's buffer capacitor against actual coin-cell sag under load.
8 — WiFi/MQTT bridge	Implement the runtime-toggled WiFi/MQTT path on the Main; confirm the ESP-NOW Remote path is unaffected whether the switch is on or off.
9 — Full bench integration	All three devices, real batteries, protoboard circuits, running for at least several days to shake out pairing/staleness/lockout edge cases before touching the fireplace.

10 — Controlled appliance test	First live connection to the fireplace under supervision — confirm on/off polarity assumptions from Section 6 against the actual unit.
11 — Schematic capture + PCB layout	Enter all three schematics in KiCad, run ERC/DRC, honor each module's antenna keep-out.
12 — Fab, assembly, bring-up	Fabricate all three boards; bring up power rails before the ESP32 modules are ever populated, same discipline as a mains board even though these are low-voltage.
13 — Enclosure	Handheld case for the Remote (battery access), small wall/shelf enclosure for the Sensor, and a Main enclosure with the fireplace terminal block and WiFi switch accessible.

15 Safety notice

This project controls a live gas flame appliance.

Even though no board in this design touches mains voltage or the gas valve directly, a bug that leaves the fireplace stuck on is a real fire and burn-injury risk, and a bug that leaves it stuck off in a safety-relied "heat the room" context is a real cold-injury risk in some use cases. Treat this as a safety-relevant system despite its low operating voltages.

- The overtemp failsafe in this design is **firmware-only**, not a hardware-independent cutout — a hung or crashed task on the Main Control Block could theoretically fail to enforce the lockout. The onboard-sensor redundancy in [Section 3](#) reduces one failure mode (a dead wireless Sensor) but does not eliminate the firmware dependency itself. If this becomes a shipped product, revisit whether a hardware comparator/thermal cutoff belongs in series with the relay drive.
- Verify the fireplace on/off polarity assumption ([Section 6](#)) on the bench, under supervision, before the first live connection to the appliance — do not assume the polarity mapping without confirming it against your specific unit's documentation or behavior.
- The de-energized relay rest state should always be the fireplace-off polarity; double-check this wiring convention specifically, since getting it backwards means firmware crashes and power loss both default to on instead of off.
- Confirm your jurisdiction's rules on unattended/automated control of gas appliances before relying on this system unsupervised — some regions have specific code requirements for automatic fireplace controls.

This document is a design plan, not a certified or inspected design. You are responsible for the safety of anything you build from it.

16 Appendix

Pin budget

Device	Count	Lines
Main Control Block	~6	I2C (SHT40) ×2, relay drive ×1, WiFi switch ×1, status LED ×1, spare/debug ×1.
Remote Control	~7	4 buttons, I2C (OLED) ×2, battery-voltage ADC ×1.
Sensor	~2	I2C (SHT40) ×2 — essentially the entire budget.

All three devices sit comfortably within their respective ESP32 variant's available GPIO with room to spare.

References

- Original design brief: `basic-design-writeup.txt` , this repository.
- ESP-IDF programming guide and component reference (Espressif).
- ESP-NOW API reference and pairing pattern discussion (Espressif ESP-IDF docs).
- Sensirion SHT4x datasheet (temperature/humidity sensor).
- TI TPS61099 synchronous boost converter datasheet.
- Diodes Inc AP63203 buck converter datasheet.
- Panasonic TQ2-5V relay datasheet.
- Prior related project: `espond32` (Noah Robertson) — reference for this document's format and for mains-isolation practices this design deliberately avoids needing.